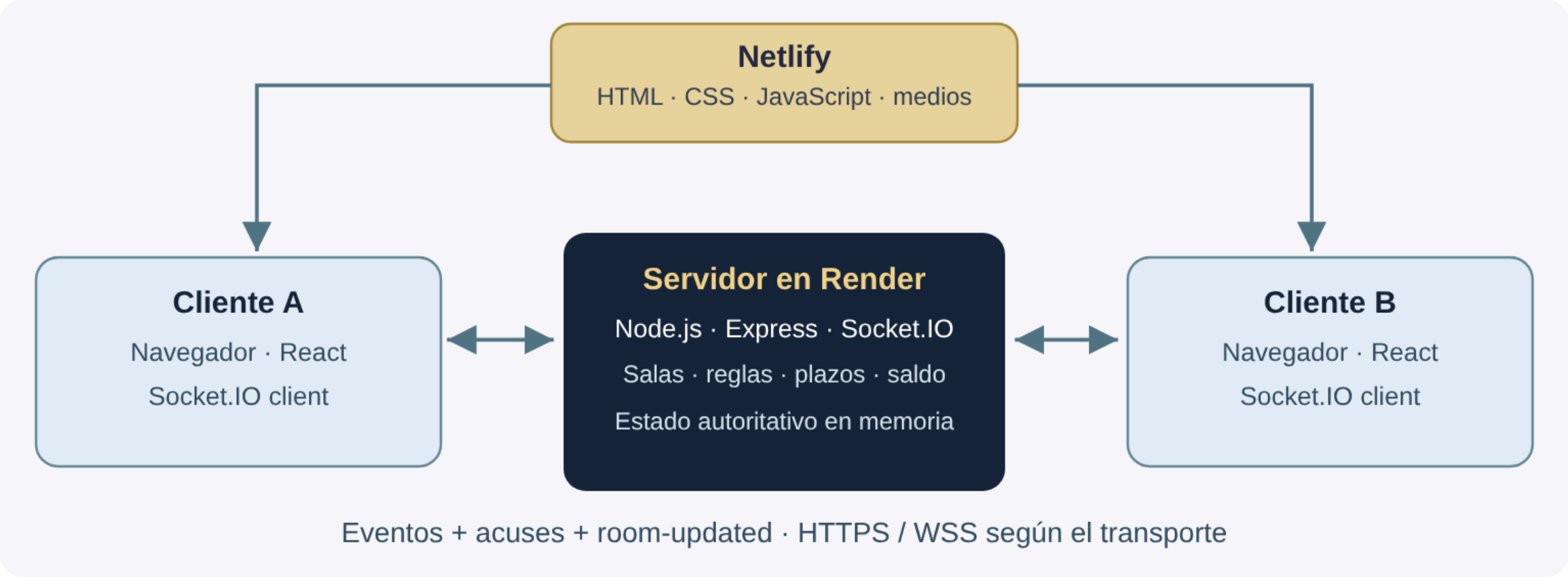
Producto final y objetivo

- Juego web 2D de tablero por turnos.
- Dos jugadores humanos conectados a una misma sala.
- Modo individual contra un bot como función adicional.
- Objetivo de red: sincronizar acciones, retos y resultados.



Sala real de dos clientes contra el servidor local.

Arquitectura cliente–servidor



Netlify entrega archivos · Render valida la partida · Socket.IO transporta los eventos

De la sala a la partida

- 1. Crear sala y recibir un código de seis caracteres.
- 2. Unirse desde un segundo cliente.
- 3. Elegir personajes distintos.
- 4. El creador inicia y el servidor distribuye el estado.

El servidor decide y sincroniza

- El cliente solicita roll-dice: no propone el resultado.
- El servidor valida el turno y genera el dado.
- También calcula posiciones, monedas, compras y victoria.
- room-updated publica una vista limitada a los participantes.

```
io.to(room.code).emit(
  "room-updated",
publicRoom(room)
);
```

Retos, plazos y privacidad

- 20 segundos para responder; 5 para ver resultados.
- Una respuesta por participante, validada en el servidor.
- El rival no ve la opción enviada antes del cierre.
- Imágenes y clips se preparan antes de iniciar el plazo.



Resultado de un duelo con dos clientes de navegador.

Estabilidad y límites actuales

- Espera de conexión de hasta 90 segundos y botón Cancelar.
- Reintentos sin duplicar la solicitud de entrada.
- Acciones fuera de turno y respuestas tardías se rechazan.
- Al desconectar se pierde la partida activa: aún no hay recuperación.

Resultados comprobados

56
lógica y conexión

38
interfaz Chromium

0
casos fallidos

- 94 casos automatizados aprobados en la versión revisada.
- Dos clientes reales de navegador; reglas con reloj simulado.
- Pantallas verticales y horizontales desde 320×568 y 740×280.
- Análisis estático y compilación aprobados.

Demostración ante la clase

- Crear una sala y compartir su código.
- Unirse, elegir personajes e iniciar desde el creador.
- Lanzar y comparar turno, posición y monedas.
- Resolver un duelo si aparece; mostrar la misma conclusión.



videojuego3d.netlify.app

Mejoras y optimizaciones futuras

- Recuperar sesiones y persistir partidas.
- Medir latencia, memoria, tráfico y carga concurrente.
- Coordinar el estado si se utilizan varios procesos.
- Enviar cambios parciales y ampliar las pruebas en dispositivos físicos.

Evidencia lista para revisión

- Informe: arquitectura, procedimientos, resultados y límites.
- Código: server/index.cjs, server/challenges.cjs y src/socket.js.
- Pruebas y evidencias incluidas con la entrega.
- El enlace público permite comprobar la interacción en red.